

PRACTICAL 7

Name: Isha Raut

Section: A4

Batch: B2

Roll No: 23

Aim: Implement Hamiltonian Cycle using Backtracking.

Problem Statement:

The Smart City Transportation Department is designing a night-patrol route for security vehicles.

Each area of the city is represented as a vertex in a graph, and a road between two areas is represented as an edge.

The goal is to find a route that starts from the main headquarters (Area A), visits each area exactly once, and returns back to the headquarters — forming a Hamiltonian Cycle.

If such a route is not possible, display a suitable message.

1) Adjacency Matrix

A B C D E

A 0 1 1 0 1

B 1 0 1 1 0

C 1 1 0 1 0

D 0 1 1 0 1

E 1 0 0 1 0

1) Adjacency Matrix

T M S H C

T 0 1 1 0 1

M 1 0 1 1 0

S 1 1 0 1 1

H 0 1 1 0 1

C 1 0 1 1 0

Code:

```
#include <stdio.h>
```

```
#define N 5
```

```
int isSafe(int v, int graph[N][N], int path[N], int pos) {
```

```
    if (graph[path[pos - 1]][v] == 0)
        return 0;
```

```
    for (int i = 0; i < pos; i++)
        if (path[i] == v)
            return 0;
```

```
    return 1;
```

```
}
```

```
int hamiltonianCycleUtil(int graph[N][N], int path[N], int pos) {
```

```
    if (pos == N) {
```

```
        if (graph[path[pos - 1]][path[0]] == 1)
```

```

        return 1;
    else
        return 0;
}

for (int v = 1; v < N; v++) {
    if (isSafe(v, graph, path, pos)) {
        path[pos] = v;

        if (hamiltonianCycleUtil(graph, path, pos + 1))
            return 1;

        path[pos] = -1;
    }
}
return 0;
}

void hamiltonianCycle(int graph[N][N], char labels[N]) {
    int path[N];

    for (int i = 0; i < N; i++)
        path[i] = -1;

    path[0] = 0;

    if (!hamiltonianCycleUtil(graph, path, 1)) {
        printf("No Hamiltonian Cycle exists.\n");
        return;
    }
}

```

```

printf("Hamiltonian Cycle exists:\n");
for (int i = 0; i < N; i++)
    printf("%c -> ", labels[path[i]]);
printf("%c\n", labels[path[0]]);
}

int main() {
    // Graph 1: A, B, C, D, E
    int graph1[N][N] = {
        {0, 1, 1, 0, 1},
        {1, 0, 1, 1, 0},
        {1, 1, 0, 1, 0},
        {0, 1, 1, 0, 1},
        {1, 0, 0, 1, 0}
    };
    char labels1[N] = {'A', 'B', 'C', 'D', 'E'};

    // Graph 2: T, M, S, H, C
    int graph2[N][N] = {
        {0, 1, 1, 0, 1},
        {1, 0, 1, 1, 0},
        {1, 1, 0, 1, 1},
        {0, 1, 1, 0, 1},
        {1, 0, 1, 1, 0}
    };
    char labels2[N] = {'T', 'M', 'S', 'H', 'C'};

    printf("Graph 1:\n");
    hamiltonianCycle(graph1, labels1);

```

```

printf("\nGraph 2:\n");
hamiltonianCycle(graph2, labels2);

return 0;
}

```

Output:

```

Graph 1:
Hamiltonian Cycle exists:
A -> B -> C -> D -> E -> A

Graph 2:
Hamiltonian Cycle exists:
T -> M -> S -> H -> C -> T

```

Competitive Code:

```

1 class Solution:
2     def check(self, n, m, edges):
3         # Build adjacency
4         adj = [[] for _ in range(n + 1)]
5         for u, v in edges:
6             adj[u].append(v)
7             adj[v].append(u)
8
9         def dfs(node, visited, count):
10            if count == n:
11                return True
12
13            for nei in adj[node]:
14                if not visited[nei]:
15                    visited[nei] = True
16                    if dfs(nei, visited, count + 1):
17                        return True
18                    visited[nei] = False
19            return False
20
21        for start in range(1, n + 1):
22            visited = [False] * (n + 1)
23            visited[start] = True
24            if dfs(start, visited, 1):
25                return 1
26        return 0

```

Problem Solved Successfully ✓

[Suggest Feedback](#)

Test Cases Passed

52 / 52

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored ⓘ

4 / 4

Your Total Score: 12 ↑

Time Taken

0.03

Link:

<https://www.geeksforgeeks.org/problems/hamiltonian-path2522/1>